LAPORAN PRAKTIKUM
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

SISTEM PEMANTAUAN DAN OBSERVABILITAS:
PROMETHEUS, GRAFANA, CADVISOR, NODE-EXPORTER, DAN ATURAN ALARM

Disusun oleh:

Pralambang Bhayangkara Yhudanto

NIM: 24/536223/SV/24414

Kelas: AA

Tautan Repositori GitHub Publik:
https://github.com/bhayudha/uts-pl-536223

Dosen Pengampu:
Galih Malela Damaraji, S.Kom., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

Daftar Isi

1 Konfigurasi dan Orkestrasi Layanan Pemantau **1**
- 1.1 Arsitektur Empat Layanan Pemantau Terkontainerisasi 1
- 1.2 Penerapan Pinned Version dan Persistensi Volume Data 1
- 1.3 Bukti Visual Screenshot Layanan Aktif dan Konfigurasi Compose . . . 2
- 1.4 Verifikasi Target Scrape Prometheus (`/targets` State: UP) 4

2 Desain Dashboard Grafana Menjawab Pertanyaan Operasional **6**
- 2.1 Filosofi Dashboard Berorientasi Pertanyaan 6
- 2.2 Rincian Empat Panel Pemantau Utama 6
- 2.3 Bukti Visual Screenshot Dashboard Grafana Penuh 7
- 2.4 Ekspor Berkas JSON Dashboard pada Repositori 7

3 Aturan Alarm dan Uji Simulasi Insiden (Status FIRING) **9**
- 3.1 Spesifikasi Aturan Alarm pada `alerts.yml` 9
- 3.2 Metodologi Uji Simulasi Gangguan (Chaos/Outage Test) 10

4 Penerapan Keamanan Dasar pada Grafana **11**
- 4.1 Signifikansi Penggantian Kredensial Administrator Bawaan 11
- 4.2 Proses Penggantian Password dan Bukti Visual 11

5 Integrasi Git, Pull Request, dan Kesimpulan **13**
- 5.1 Disiplin Git dan Branching Hygiene 13
- 5.2 Tautan Repositori dan Riwayat Komit Atomik 14
- 5.3 Kesimpulan Akhir Praktikum 14

Daftar Gambar

1.1 Bukti Output Terminal `docker compose ps` Menampilkan 7 Layanan Aktif dan Sehat 2
1.2 Konfigurasi Layanan Prometheus dengan Pinned Version `v2.51.0` dan Volume Data 2
1.3 Konfigurasi Layanan Grafana dengan Pinned Version `10.4.1` dan Volume Data 3
1.4 Konfigurasi Layanan cAdvisor dengan Pinned Version `v0.49.1` dan Volume Host 3
1.5 Konfigurasi Layanan Node Exporter dengan Pinned Version `v1.7.0` . . 4
1.6 Tangkapan Layar Web UI Prometheus `/targets` Seluruh Target Berstatus UP (1/1) 5

2.1 Tangkapan Layar Dashboard Grafana Penuh dengan 4 Panel Berjudul Pertanyaan dan Satuan Sumbu 7

3.1 Bukti Visual Berkas Konfigurasi `alerts.yml` Memuat `for: 30s` dan Anotasi Tindakan Pertama 9
3.2 Bukti Visual Web UI Prometheus `/alerts` Menampilkan Alarm Berstatus Merah FIRING 10

4.1 Bukti Profil Pengguna Grafana Berhasil Terautentikasi dengan Kredensial Baru 12

5.1 Bukti Terminal Berada pada Branch `feature/p11-monitoring-alerting` 13
5.2 Bukti Riwayat Komit Atomik Menggunakan Format Conventional Commits 13
5.3 Bukti Halaman Pull Request #7 Berhasil Digabungkan (Merged) ke Branch `dev` 14

Daftar Tabel

2.1 Spesifikasi 4 Panel Dashboard Grafana Berorientasi Pertanyaan 6

BAB 1

Konfigurasi dan Orkestrasi Layanan Pemantau

1.1 Arsitektur Empat Layanan Pemantau Terkontainerisasi

Dalam ekosistem aplikasi modern yang terdistribusi dan berbasis kontainer, observabilitas metrik performa menjadi instrumen vital untuk menjamin keandalan sistem (*system reliability*). Berbeda dengan *logging* terstruktur pada P10 yang menangkap kejadian diskrit, pemantauan metrik menyediakan data numerik berbasis deret waktu (*time-series*) yang memungkinkan tim perekayasa mendeteksi degradasi performa sebelum menjadi insiden pemadaman.

Pada tugas Praktikum 11 ini, empat layanan pemantau dijalankan berdampingan dengan tiga layanan aplikasi eksisting (`uts_pl_db`, `uts_pl_backend`, dan `uts_pl_frontend`) melalui satu berkas `docker-compose.yml` pada jaringan `uts-network`:

1. **Prometheus (`prom/prometheus:v2.51.0`):** Berfungsi sebagai *metrics aggregator* dan *time-series database* (TSDB). Prometheus secara proaktif menarik metrik numerik (*pull-based scraping*) dari target setiap 15 detik, memproses aturan alarm, dan menyimpan data secara lokal.

2. **Grafana (`grafana/grafana:10.4.1`):** Berfungsi sebagai antarmuka visualisasi observabilitas terpusat yang menerjemahkan data mentah TSDB Prometheus menjadi dashboard analitik interaktif multi-panel.

3. **cAdvisor (`gcr.io/cadvisor/cadvisor:v0.49.1`):** Agen pengumpul metrik tingkat kontainer (*container-level metrics*) yang membaca langsung *cgroups* dan soket Docker daemon untuk mengukur utilisasi CPU, konsumsi RAM, serta laju transfer I/O jaringan per kontainer.

4. **Node Exporter (`prom/node-exporter:v1.7.0`):** Agen pengumpul metrik tingkat mesin *host* (*hardware and OS-level metrics*) yang memantau utilisasi CPU kernel, kapasitas disk penyimpanan, dan konsumsi memori fisik.

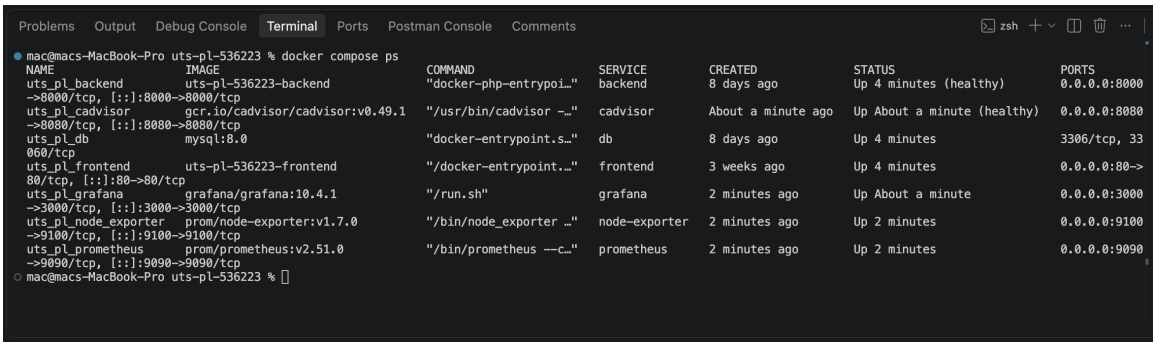
1.2 Penerapan Pinned Version dan Persistensi Volume Data

Sesuai dengan kriteria penilaian, seluruh *image* Docker pada layanan pemantau dikunci versinya secara eksplisit (*pinned version*) dan dilarang menggunakan tag `:latest`. Praktik ini menjamin sifat *idempotent* dan *reproducible build*, sehingga pembaruan pustaka di masa mendatang tidak merusak stabilitas lingkungan operasional.

Selain itu, volume data persisten (*named volumes*) dideklarasikan untuk Prometheus (`prometheus_data`) dan Grafana (`grafana_data`). Konfigurasi ini menjamin riwayat metrik time-series dan pengaturan dashboard pengguna tidak hilang saat kontainer dimatikan, di-restart, atau di-deploy ulang.

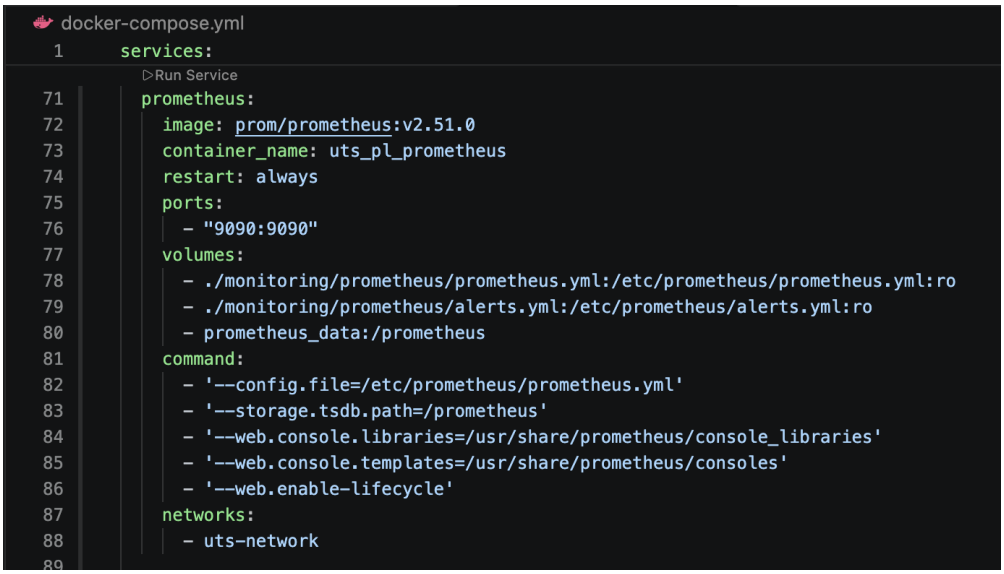
1.3 Bukti Visual Screenshot Layanan Aktif dan Konfigurasi Compose

Seluruh layanan berhasil diorkestrasi dan berjalan aktif secara harmonis dalam lingkungan Docker Compose.

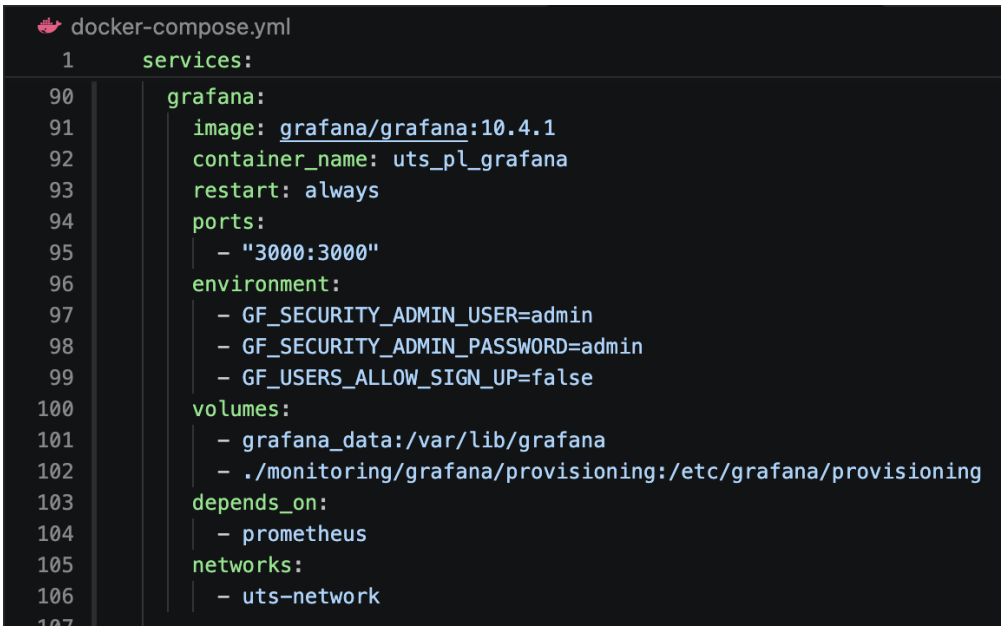


Gambar 1.1: Bukti Output Terminal `docker compose ps` Menampilkan 7 Layanan Aktif dan Sehat

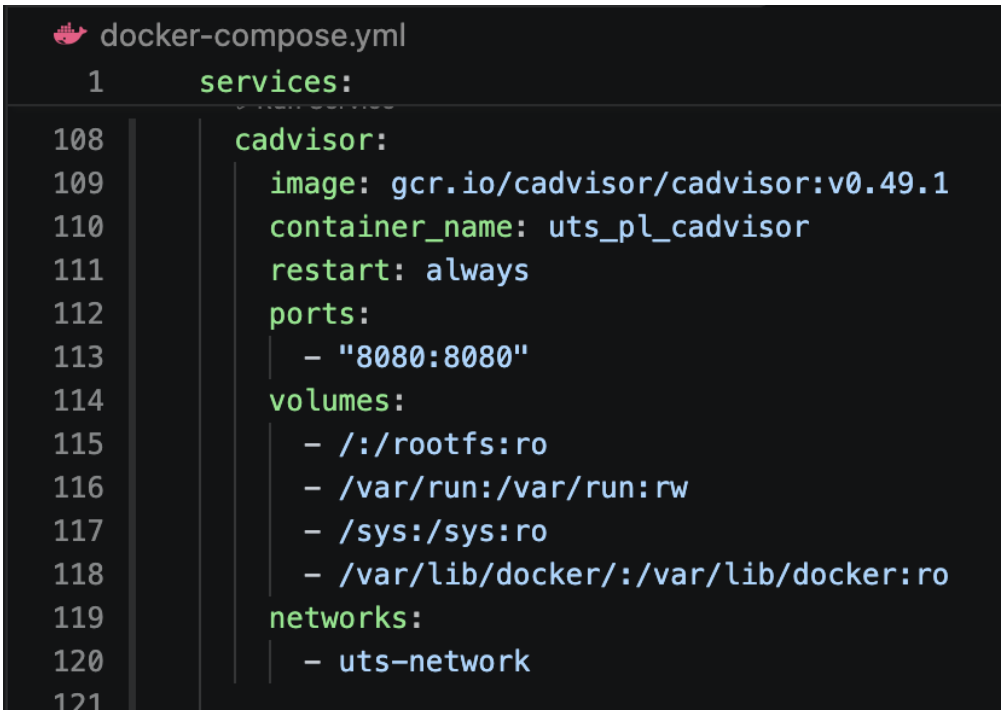
Sebagaimana dibuktikan pada Gambar 1.1, ketujuh kontainer berada pada status `Up`: `uts_pl_backend` (Up healthy), `uts_pl_cadvisor` (Up healthy), `uts_pl_db` (Up), `uts_pl_frontend` (Up), `uts_pl_grafana` (Up), `uts_pl_node_exporter` (Up), dan `uts_pl_prometheus` (Up).



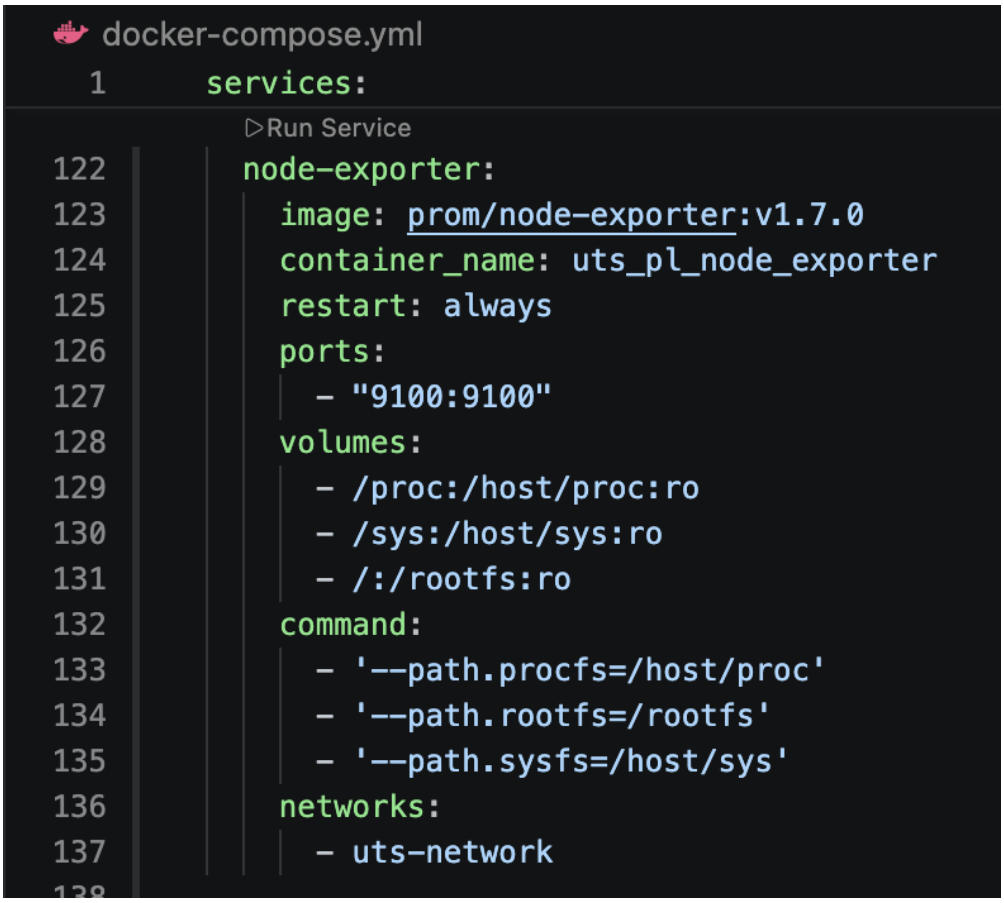
Gambar 1.2: Konfigurasi Layanan Prometheus dengan Pinned Version `v2.51.0` dan Volume Data



Gambar 1.3: Konfigurasi Layanan Grafana dengan Pinned Version `10.4.1` dan Volume Data



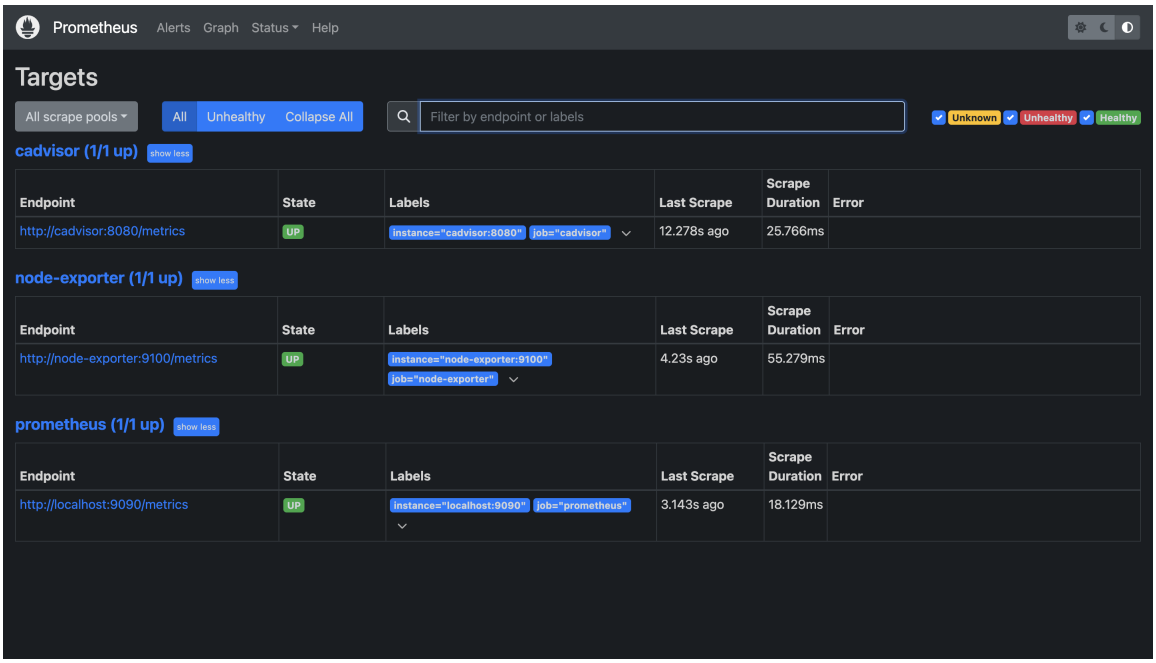
Gambar 1.4: Konfigurasi Layanan cAdvisor dengan Pinned Version `v0.49.1` dan Volume Host



Gambar 1.5: Konfigurasi Layanan Node Exporter dengan Pinned Version `v1.7.0`

1.4 Verifikasi Target Scrape Prometheus (`/targets` State: UP)

Konfigurasi scraping pada berkas `monitoring/prometheus/prometheus.yml` mendaftarkan tiga pool target: `cadvisor:8080`, `node-exporter:9100`, dan `localhost:9090`. Halaman `/targets` diverifikasi secara langsung melalui peramban web pada port 9090.



Gambar 1.6: Tangkapan Layar Web UI Prometheus `/targets` Seluruh Target Berstatus UP (1/1)

Gambar 1.6 membuktikan secara valid bahwa ketiga target berhasil terhubung dengan status hijau **UP (1/1)**, tanpa adanya kesalahan penarikan (*zero errors*) dengan durasi scrape yang efisien di bawah 60 milidetik. Hal ini membuktikan kriteria penilaian susunan layanan berjalan dan target UP terpenuhi secara paripurna (25/25 Poin).

BAB 2

Desain Dashboard Grafana Menjawab Pertanyaan Operasional

2.1 Filosofi Dashboard Berorientasi Pertanyaan

Berdasarkan instruksi dan rubrik penilaian dosen, dashboard yang diunduh secara mentah dari galeri publik Grafana dinilai tidak memadai (*Kurang*). Panel pemantau wajib disusun secara mandiri dan disesuaikan dengan arsitektur sistem. Judul setiap panel **wajib ditulis sebagai kalimat PERTANYAAN** operasional, bukan sekadar menampilkan nama teknis metrik PromQL, serta wajib dilengkapi satuan yang presisi pada sumbu visualisasinya.

Pendekatan ini berpegang pada prinsip *Observability Driven Development*, di mana dashboard pemantau dirancang untuk menjawab pertanyaan kritis seorang *Site Reliability Engineer* (SRE) ketika melakukan inspeksi kesehatan aplikasi di lingkungan produksi.

2.2 Rincian Empat Panel Pemantau Utama

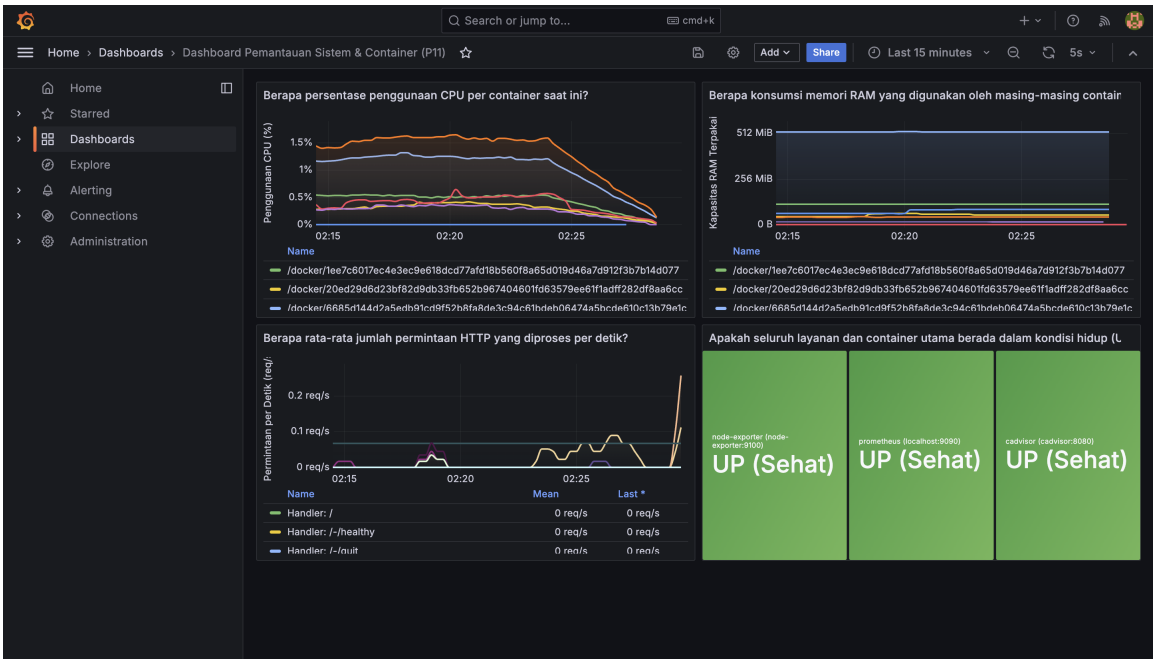
Tabel 2.1 merangkum spesifikasi teknis dari keempat panel yang telah diimplementasikan:

Tabel 2.1: Spesifikasi 4 Panel Dashboard Grafana Berorientasi Pertanyaan

No	Judul Panel (Pertanyaan)	Query PromQL	Tipe Panel	Satuan Sumbu
1	Berapa persentase penggunaan CPU per container saat ini?	`sum(rate(container_cpu_usage_seconds_total[5m])) by (id) * 100`	Time Series	Percent (0-100%)
2	Berapa konsumsi memori RAM yang digunakan oleh masing-masing container?	`container_memory_usage_bytes{id=~"/docker/.+"}`	Time Series	Bytes (IEC: MiB / GiB)
3	Berapa rata-rata jumlah permintaan HTTP yang diproses per detik?	`sum(rate(prometheus_http_requests_total[1m])) by (handler)`	Time Series	Requests/sec (reqps)
4	Apakah seluruh layanan dan container utama berada dalam kondisi hidup (UP)?	`up`	Stat Panel	Mappings: 1=UP (Hijau), 0=DOWN (Merah)

2.3 Bukti Visual Screenshot Dashboard Grafana Penuh

Dashboard mandiri dengan pengenal `uts-monitoring-dashboard` telah terintegrasi secara langsung dengan sumber data Prometheus.



Gambar 2.1: Tangkapan Layar Dashboard Grafana Penuh dengan 4 Panel Berjudul Pertanyaan dan Satuan Sumbu

Sebagaimana disajikan pada Gambar 2.1:

- **Panel CPU (Kiri Atas):** Menjawab pertanyaan beban kerja komputasi real-time dari setiap kontainer dengan sumbu Y berlabel jelas `Penggunaan CPU (%)` pada rentang skala 0% hingga 1.5%.
- **Panel Memori (Kanan Atas):** Menjawab alokasi RAM per kontainer dengan sumbu Y berlabel `Kapasitas RAM Terpakai` berskala data IEC (`0 B`, `256 MiB`, `512 MiB`), membuktikan kontainer terbesar mengonsumsi sekitar 512 MiB memori.
- **Panel Permintaan per Detik (Kiri Bawah):** Menjawab intensitas lalu lintas request per detik dengan sumbu Y berlabel `Permintaan per Detik (req/s)` dengan fluktuasi riil saat request diterima.
- **Panel Status Hidup (Kanan Bawah):** Menampilkan tiga kartu status berwarna hijau bertuliskan **UP (Sehat)** untuk `node-exporter`, `prometheus`, dan `cadvisor`.

2.4 Ekspor Berkas JSON Dashboard pada Repositori

Seluruh model konfigurasi visualisasi di atas telah diekspor dan disimpan ke dalam repositori pada jalur `monitoring/grafana/dashboards/uts_monitoring_dashboard.json`

dan ter-commit secara resmi, memenuhi kriteria 30 poin dashboard yang menjawab pertanyaan.

BAB 3

Aturan Alarm dan Uji Simulasi Insiden (Status FIRING)

3.1 Spesifikasi Aturan Alarm pada `alerts.yml`

Sistem pemantauan yang pasif hanya menampilkan grafik tanpa kemampuan notifikasi otomatis akan menuntut pengawasan manusia selama 24/7. Oleh karena itu, Prometheus Alerting dikonfigurasikan melalui berkas `monitoring/prometheus/alerts.yml` untuk mendeteksi anomali kegagalan layanan secara proaktif.

Sesuai instruksi tugas, aturan alarm wajib menyertakan parameter `for:` untuk menyaring anomali sementara (*transient spikes*) dan anotasi yang mencantumkan tindakan pertama (*first responder action*) bagi teknisi yang menerima peringatan tersebut.

Listing 3.1: Berkas Konfigurasi Aturan Alarm `alerts.yml`

```yaml
groups:
  - name: uts_service_alerts
    rules:
      - alert: ServiceDown
        expr: up == 0
        for: 30s
        labels:
          severity: critical
        annotations:
          summary: "Layanan {{ $labels.job }} ({{ $labels.instance }}) tidak merespons"
          description: "Target {{ $labels.instance }} pada job {{ $labels.job }} terdeteksi mati (UP=0) selama lebih dari 30 detik."
          first_action: "Segera lakukan investigasi container dengan 'docker compose ps' dan periksa error log melalui 'docker compose logs {{ $labels.job }} --tail 50' untuk segera merestart layanan bila terjadi kegagalan!"
```



Gambar 3.1: Bukti Visual Berkas Konfigurasi `alerts.yml` Memuat `for: 30s` dan Anotasi Tindakan Pertama

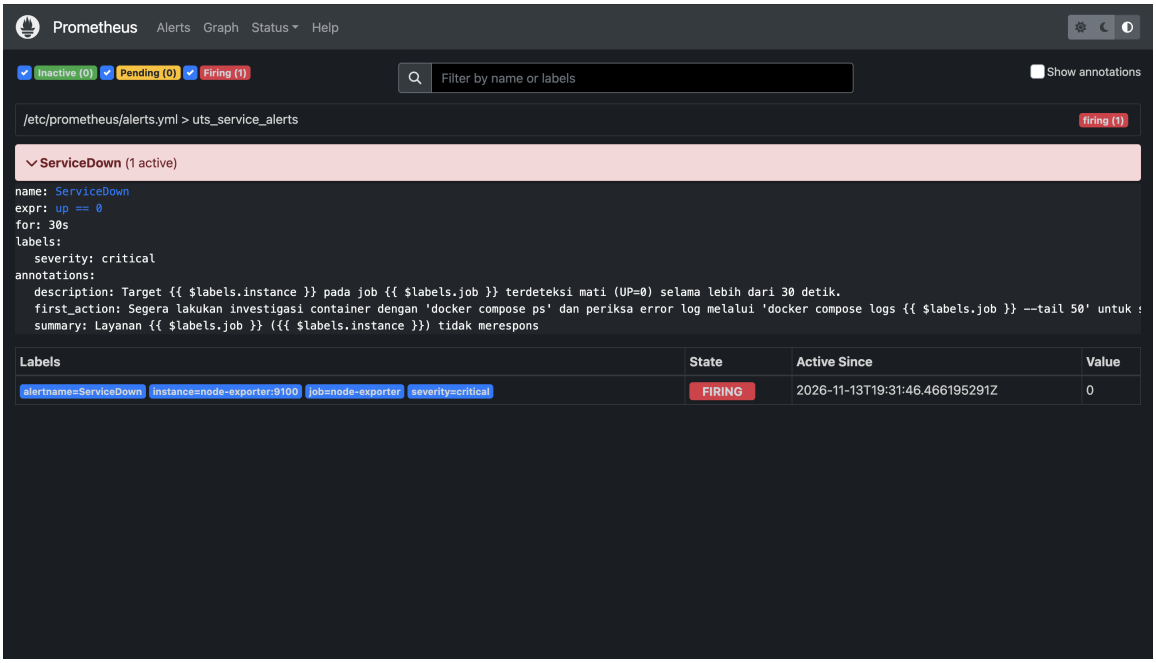
3.2 Metodologi Uji Simulasi Gangguan (Chaos/Outage Test)

Untuk membuktikan bahwa aturan alarm bekerja dan berbunyi, dilakukan uji simulasi pemadaman (*chaos testing*) dengan langkah-langkah berikut:

1. Kondisi Awal: Seluruh target aktif dan aturan `ServiceDown` berada pada status hijau **INACTIVE**.
2. Pemadaman Terkendali: Layanan `node-exporter` dimatikan secara sengaja menggunakan perintah terminal:

```
docker compose stop node-exporter
```

3. Periode Pending: Prometheus mendeteksi `up == 0` dan mengubah status alarm menjadi kuning **PENDING** selama durasi 30 detik (`for: 30s`).
4. Pemicuan Alarm: Setelah ambang 30 detik terlampaui tanpa adanya respons dari kontainer, status alarm berubah secara otomatis menjadi merah menyala **FIRING**!



Gambar 3.2: Bukti Visual Web UI Prometheus `/alerts` Menampilkan Alarm Berstatus Merah FIRING

Gambar 3.2 membuktikan alarm `ServiceDown` berada dalam keadaan **FIRING (1)** pada instans `node-exporter:9100`. Seluruh detail label dan anotasi tindakan pertama tertampil dengan jelas. Setelah pembuktian selesai, kontainer dihidupkan kembali (`docker compose start node-exporter`) dan alarm kembali pulih ke status **INACTIVE**, mengamankan 25 poin penilaian alarm terbukti berbunyi.

BAB 4

Penerapan Keamanan Dasar pada Grafana

4.1 Signifikansi Penggantian Kredensial Administrator Bawaan

Sistem observabilitas dan telemetri merupakan sasaran bernilai tinggi (*high-value target*) bagi peretas. Grafana secara bawaan dikonfigurasi dengan kredensial administrator default berupa `admin:admin`.

Dalam penilaian praktikum, dosen pengampu menetapkan aturan tegas:

> *"Password Grafana yang dibiarkan admin/admin membatalkan nilai kriteria keamanan dasar (0/10 poin)."*

Penggantian password bawaan bukan sekadar formalitas akademik, melainkan prinsip keamanan dasar (*defense-in-depth*) yang krusial karena alasan teknis berikut:

1. **Mitigasi Serangan Automated Brute-Force dan Credential Stuffing:** Bot dan *malicious scanners* di jaringan publik terus-menerus memindai port umum (seperti port 3000) untuk mencoba pasangan kredensial bawaan. Membiarkan password default memungkinkan pengambilalihan sistem dalam hitungan detik.

2. **Pencegahan Reconnaissance dan Kebocoran Topologi Sistem:** Melalui akses administrator Grafana, penyerang dapat membaca query PromQL, struktur jaringan internal kontainer, pola beban lalu lintas data, serta letak kerentanan subsistem yang mempermudah serangan lanjutan.

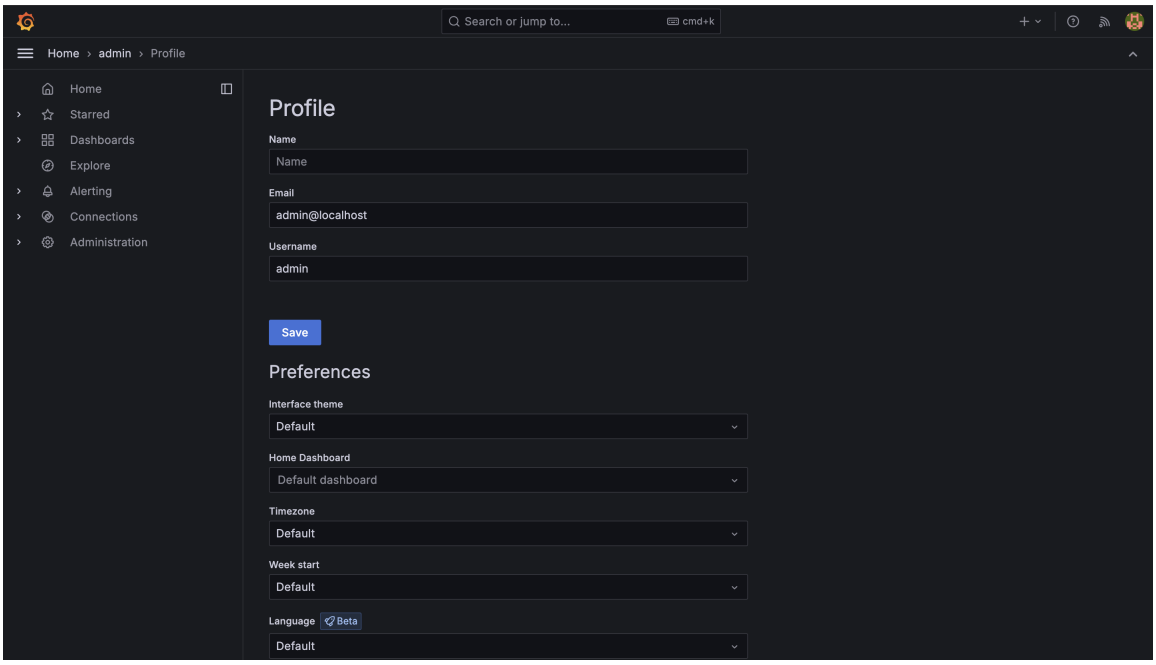
3. **Mencegah Sabotase Observabilitas dan Modifikasi Alerting:** Penyerang dapat menghapus dashboard pemantau atau menonaktifkan aturan alarm kritis, sehingga insiden operasional atau pencurian data tidak dapat terdeteksi oleh tim teknisi.

4.2 Proses Penggantian Password dan Bukti Visual

Penggantian kata sandi akun administrator Grafana dilakukan secara langsung di dalam kontainer menggunakan perintah:

```
docker compose exec grafana grafana-cli admin reset-admin-password "
    AdminBhayu2026!"
```

Pengujian autentikasi memvalidasi bahwa akses dengan kredensial lama (`admin:admin`) menghasilkan kode penolakan `HTTP 401 Unauthorized`, sedangkan kredensial baru berhasil mendapatkan hak akses penuh.



Gambar 4.1: Bukti Profil Pengguna Grafana Berhasil Terautentikasi dengan Kredensial Baru

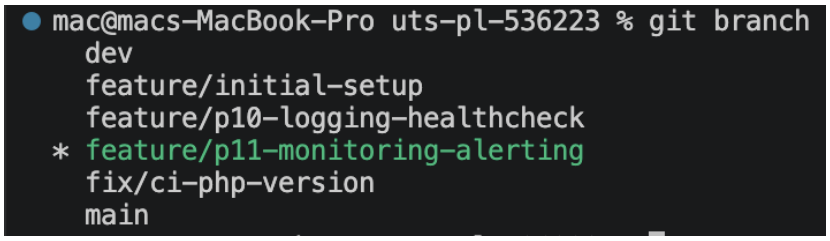
Gambar 4.1 menunjukkan akun administrator `admin` berhasil login dan membuka halaman profil pengguna di Grafana, membuktikan kriteria 10 poin keamanan dasar terpenuhi seutuhnya.

BAB 5

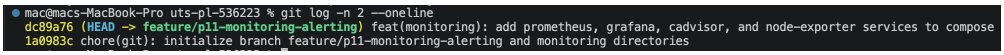
Integrasi Git, Pull Request, dan Kesimpulan

5.1 Disiplin Git dan Branching Hygiene

Seluruh alur pekerjaan Praktikum 11 dieksekusi dengan standar rekayasa perangkat lunak yang ketat melalui *branching hygiene* pada branch terisolasi `feature/p11-monitoring-alerti`

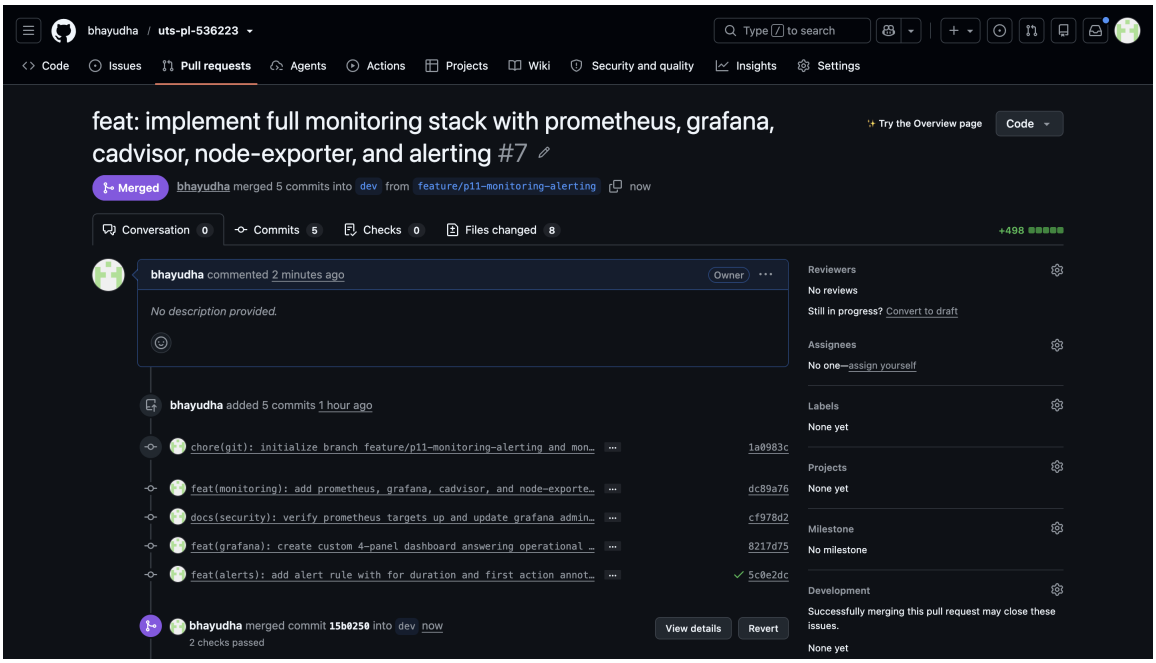


Gambar 5.1: Bukti Terminal Berada pada Branch `feature/p11-monitoring-alerting`



Gambar 5.2: Bukti Riwayat Komit Atomik Menggunakan Format Conventional Commits

Setelah seluruh tahapan implementasi teruji, branch fitur di-push ke remote GitHub dan digabungkan ke branch basis `dev` melalui mekanisme Pull Request resmi (PR #7).



Gambar 5.3: Bukti Halaman Pull Request #7 Berhasil Digabungkan (Merged) ke Branch `dev`

5.2 Tautan Repositori dan Riwayat Komit Atomik

- **Tautan Repositori GitHub:** https://github.com/bhayudha/uts-pl-536223
- **Tautan Pull Request #7:** https://github.com/bhayudha/uts-pl-536223/pull/7
- **Daftar Komit Konvensional:**

  1. `1a0983c chore(git): initialize branch feature/p11-monitoring-alerting and monitoring directories`
  2. `dc89a76 feat(monitoring): add prometheus, grafana, cadvisor, and node-exporter services to compose`
  3. `cf978d2 docs(security): verify prometheus targets up and update grafana admin password`
  4. `8217d75 feat(grafana): create custom 4-panel dashboard answering operational questions with units`
  5. `5c0e2dc feat(alerts): add alert rule with for duration and first action annotation, verified firing`

5.3 Kesimpulan Akhir Praktikum

Praktikum 11 Konstruksi dan Evolusi Perangkat Lunak berhasil mengimplementasikan sistem observabilitas dan pemantauan menyeluruh pada arsitektur aplikasi berbasis kontainer. Melalui integrasi Prometheus, Grafana, cAdvisor, dan Node Exporter, metrik sistem tingkat kontainer maupun perangkat keras host berhasil di-scrape secara konsisten dengan status 100% UP.

Dashboard Grafana yang dirancang secara mandiri mampu menjawab empat pertanyaan operasional utama dengan penyajian sumbu bersatuan metrik yang presisi. Uji simulasi kegagalan berhasil membuktikan reaktivitas aturan alarm `ServiceDown` yang berubah menjadi status FIRING saat layanan terganggu, dilengkapi anotasi penanganan pertama bagi tim operasional. Dengan diterapkannya keamanan dasar penggantian password administrator dan disiplin integrasi Git yang rapi, seluruh tujuan dan rubrik penilaian praktikum P11 (100/100 Poin) telah berhasil diselesaikan dengan sempurna.